

Base de la POO en Python

Mathieu RAYNAL

mathieu.raynal@irit.fr

<http://www.irit.fr/~Mathieu.Raynal>



Déclarer une classe

- Déclarer une classe

```
class MaClasse:  
    "Description de ma classe ..."  
  
    ...
```

Constructeur

- Constructeur

```
def __init__(self [,params]):  
    ...
```

- Il ne peut y avoir qu'un seul constructeur dans la classe
 - Possibilité de mettre des valeurs par défaut aux paramètres

```
def __init__(self, x=0, y=0):  
    ...
```

Méthode de classe

- Pour « simuler » d'autres constructeurs

```
@classmethod  
def nom_de_la_methode(cls[, params]):  
    return cls(...)
```

Attributs et méthodes d'une classe

- Attribut (variable d'instance)
 - Généralement déclaré et initialisé dans le constructeur

```
self.monAttribut = saValeur
```

- Les méthodes se déclarent comme les fonctions
 - Obligatoirement un premier paramètre nommé ***self***

```
def maMethode(self [,params]):
```

Objet : Création d'une instance d'une classe

```
class Point:
    "Définition d'un point en 2D"

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    @classmethod
    def creer_point(cls, y):
        return cls(0, y)

    def deplacer(self, dx, dy):
        self.x += dx
        self.y += dy

p = Point(1, 2)
p2 = Point()
p3 = Point.creer_point(4)
```

Utilisation d'un membre depuis un objet

- Membre = attribut ou méthode

```
p.x = 5  
p2.déplacer(3, 4)
```

- Afficher la description d'un objet

```
print(p. __doc__)
```

Les méthodes `__str__` et `__repr__`

- Pour afficher la représentation d'un objet, il faut définir une des méthodes suivantes

```
def __str__(self):
```

- Ou

```
def __repr__(self):
```

- Ces méthodes doivent retourner la chaîne de caractères que l'on souhaite afficher

Les énumérations

- Déclaration d'une énumération

```
from enum import Enum, auto

class Color(Enum):
    RED = auto()
    GREEN = auto()
    BLUE = auto()
```

- Utilisation d'une énumération

```
c = Color.BLUE
if c == Color.BLUE:
    print('OK')
```

- Parcours des valeurs d'une énumération

```
for c in Color:
    print(f"{c.name} a pour valeur {c.value}")
```

Match / case

```
day = "Monday"

# Match the day to predefined patterns
match day:
    case "Saturday" | "Sunday":
        print(f"{day} is a weekend.")
    case "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday":
        print(f"{day} is a weekday.")
    case _:
        print("That's not a valid day of the week.")
```

Surcharge d'opérateur

Opérateurs mathématiques

Méthode	Opérateur
<code>__add__</code>	<code>+</code>
<code>__sub__</code>	<code>-</code>
<code>__mul__</code>	<code>*</code>
<code>__truediv__</code>	<code>/</code>
<code>__mod__</code>	<code>%</code>
<code>__power__</code>	<code>**</code>

Opérateurs de comparaison

Méthode	Opérateur
<code>__eq__</code>	<code>==</code>
<code>__ne__</code>	<code>!=</code>
<code>__gt__</code>	<code>></code>
<code>__ge__</code>	<code>>=</code>
<code>__lt__</code>	<code><</code>
<code>__le__</code>	<code><=</code>

```
def __add__(self, p):
    return Point(self.x+p.x, self.y+p.y)
```

```
p1 = Point(3,2)
p2 = Point(4,5)
p = p1 + p2
```

Exercice

- Dans la classe Vecteur réalisée précédemment en TP
 - Ajouter les surcharges d'opérateurs de + et –
 - Remplacer produit_vectoriel par la surcharge d'opérateurs *
 - Créer les surcharges d'opérateurs pour tester les égalités et inégalités entre 2 vecteurs

Visibilité des membres

- Les membres peuvent être
 - publics
 - Accès possible depuis une instance de la classe
 - protégés : nom débute par `_`
 - Accès qu'à l'intérieur de la classe ou depuis une classe fille
 - privés : nom débute par `__`
 - Accès qu'à l'intérieur de la classe

Les getters (accesseurs) et setters (mutateur)

- Dans les deux cas, le nom de la méthode est le nom de l'attribut à modifier
 - Pour l'accesseur, faire précéder la méthode de *@property*

```
@property  
def x(self):  
    return self.__x
```

- Pour le mutateur, faire précéder la méthode de *@nomAttribut.setter*

```
@x.setter  
def x(self, x):  
    self.__x = x
```

Exercice

- Dans la classe Vecteur
 - Mettre les attributs x, y, z en privé
 - Modifier la méthode norme() pour que celle-ci ne renvoie plus le résultat mais le stocke dans un attribut norme lui-même privé
 - Ajouter les accesseurs et les mutateurs pour x, y et z de telle sorte à mettre à jour la norme à chaque modification

Héritage

- Une classe qui hérite d'une autre aura (*a minima*)
 - Les mêmes attributs
 - Les mêmes méthodes

à Il n'est pas nécessaire de les définir dans la classe. Elle en **hérite** automatiquement
- Elle peut avoir des attributs et/ou méthodes supplémentaires qui lui seront propre
- Une classe peut redéfinir une méthode déjà présente dans la classe dont elle hérite à on parle de **masquage**

Héritage

- Déclaration de la classe dont on hérite

```
class MaClasse(ClasseMere):  
    ...
```

- Utilisation de méthodes de la classe mère

```
super().laMethodeClasseMere()
```

Exemple d'héritage

```
class Point2D:
    def __init__(self, nom, x, y):
        self.nom = nom
        self.x = x
        self.y = y

    def get_nom(self):
        return self.nom

    def deplacer(self, x, y):
        self.x += x
        self.y += y

    def distance(self, other):
        return ((self.x - other.x) ** 2
                + (self.y - other.y) ** 2) ** 0.5

    def __str__(self):
        return f"({self.x}, {self.y})"
```

```
class Point3D(Point2D):
    def __init__(self, nom, x, y, z):
        super().__init__(nom, x, y)
        self.z = z

    def deplacer(self, x, y, z):
        super().deplacer(x, y)
        self.z += z

    def distance(self, other):
        return ((self.x - other.x) ** 2
                + (self.y - other.y) ** 2
                + (self.z - other.z) ** 2) ** 0.5

    def __str__(self):
        return f"({self.x}, {self.y}, {self.z})"
```

Exemple d'héritage

```
class Mutation:  
    def appliquer(self, adn):  
        pass
```

```
class Substitution(Mutation):  
    def appliquer(self, adn):  
        pos = random.randint(0, len(adn.sequence) - 1)  
        base = random.choice("ATCG")  
        seq = list(adn.sequence)  
        seq[pos] = base  
        adn.sequence = "".join(seq)  
  
class Insertion(Mutation):  
    def appliquer(self, adn):  
        pos = random.randint(0, len(adn.sequence))  
        base = random.choice("ATCG")  
        adn.sequence = adn.sequence[:pos] + base + adn.sequence[pos:]  
  
class Deletion(Mutation):  
    def appliquer(self, adn):  
        if len(adn.sequence) > 1:  
            pos = random.randint(0, len(adn.sequence) - 1)  
            adn.sequence = adn.sequence[:pos] + adn.sequence[pos+1:]
```

Exemple d'héritage (suite)

```
class Cellule:
    def __init__(self, adn):
        self.adn = adn
        self.fitness = self.evaluer_fitness()

    def evaluer_fitness(self):
        # Exemple : plus il y a de "A", plus c'est adapté
        return self.adn.sequence.count("A")

    def muter(self, mutation):
        mutation.appliquer(self.adn)
        self.fitness = self.evaluer_fitness()
```

```
def mutation(self, taux):
    mutations = [Substitution(), Insertion(), Deletion()]
    for cellule in self.cellules:
        if random.random() < taux:
            mutation = random.choice(mutations)
            cellule.muter(mutation)
```